

Wizard's Chess System Specification

Comprehensive Technical Architecture for Voice-Controlled Hybrid Physical & 3D Chess

PROJECT SCOPE

Full System Architecture

INTERFACE PARADIGM

Voice & Electro-Mechanics

DOCUMENT VERSION

v1.0 • Final Draft

1. System Executive Overview

The Wizard's Chess system is a dual-interface chess platform combining real-time speech recognition, 3D animated web rendering, and an automated mechatronic physical chessboard. Players issue spoken move commands (e.g., "Knight to F3"), which are processed through a phonetic normalization layer and validated by an engine core before translating into simultaneous physical motor operations and animated visual rendering.

SYSTEM DATA FLOW PIPELINE

```
[ Voice Input ] —▶ [ Web Speech API / Parser ] —▶ [ chess.js State Engine ]
|
├── [ 3D Web UI: Three.js + GSAP ] [ Hardware: WebSockets / Serial ]
│   ├── Piece Arc Interpolation ─ Microcontroller (ESP32/Arduino)
│   ├── Shatter/Dust Particle Effects ─ CoreXY Stepper Gantry Movement
│   └── Audio FX & Voice Feedback ─ 12V Electromagnet & LED Grid
```

2. Voice Interface & Command Disambiguation

Voice recognition presents challenges regarding phonetic misinterpretation of chess algebraic coordinates. The speech recognition pipeline utilizes a fuzzy phonetic parser to map continuous audio transcriptions into standardized SAN notation.



Speech Recognition

Uses Web Speech API or local Whisper models. Captures continuous audio streams, returning raw transcriptions for command evaluation.



Phonetic Mapping

Resolves ambiguous terms: "Night" → Knight, "See" → C-file, "Rock/Rook" → Rook, "To/Two/Too" → 2. Normalizes phonetic overlaps.



Legal Move Validation

Executes logic via chess.js. Cross-references parsed commands against all legal moves for the current board state.



Audio Disambiguation

If a command is ambiguous (e.g., "Rook to d1" with two valid rooks), voice synthesis prompts: "Which rook to d1?"

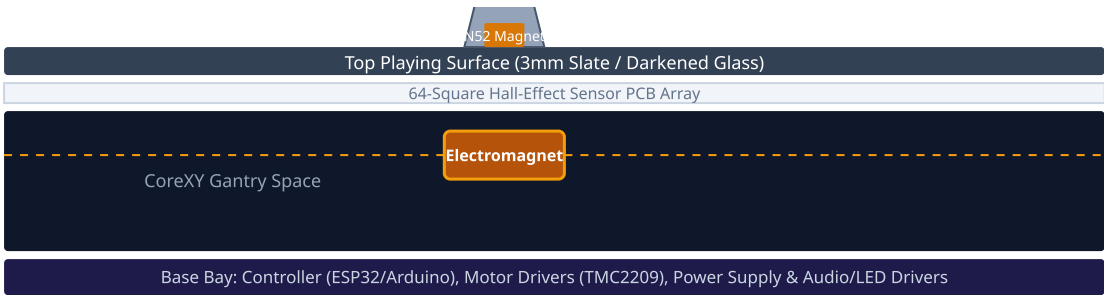
Command Grammar Standard

Accepts flexible phrasing variants: "Knight to f3", "Pawn e2 to e4", "Pawn takes d5", and "Castle kingside". Invalid moves trigger spoken error responses without altering board state.

3. Physical Board Mechatronics Engineering

The physical board utilizes a silent, under-board CoreXY gantry mechanism. An electromagnet drives carved marble pieces equipped with embedded neodymium bases without visible external mechanisms.

Physical Enclosure Cross-Section



4. Hardware Subsystem Specifications

Subsystem	Component Selection	Functional Role
Motion Mechanism	CoreXY Belt Frame + Dual NEMA 17 Steppers	Provides smooth $\$X\$$ and $\$Y\$$ positioning under the board grid using silent stepper drivers.
Magnetic Coupling	12V 5kg Holding Force Electromagnet	Grips the N52 neodymium base of pieces through the board during movement sequences.
State Detection	8x8 Hall-Effect Sensor Matrix Array	Detects piece presence across all 64 squares for manual setup and move verification.
Microcontroller	ESP32 / Raspberry Pi Pico	Receives target coordinates from web app over Serial/WebSocket and executes motor paths.
Feedback Systems	RGB Addressable LEDs & 10W Sound Transducer	Provides visual flares beneath target squares and physical stone clatter audio feedback.

5. Pathfinding & Seam Routing Algorithm

To navigate pieces across occupied tiles without collisions, the software executes pathfinding along the seam lines separating the squares ($\$0.5\$$ -square offset alignment):

- Step 1 (Engagement):** The electromagnet homes beneath the piece and energizes.
- Step 2 (Offset Alignment):** The piece is shifted 0.5 squares onto the nearest seam line.
- Step 3 (Traversal):** The piece moves along grid seams around occupied squares to its destination coordinate.
- Step 4 (Settling):** The piece aligns centered on the target square, and the electromagnet de-energizes.

6. Carved Marble Piece Specifications

Physical pieces are carved from marble or cast resin mixed with stone powder to balance weight with an authentic appearance.

Internal Cavity

A counter-bored cavity in the base houses a high-grade **N52 Neodymium disc magnet** (15mm x 3mm), sealed with epoxy.

Low-Friction Pad

The base contact layer is covered with smooth PTFE (Teflon) tape, reducing friction to permit smooth sliding over stone or glass.

7. 3D Web App & Capture Visual Effects

The web application mirrors physical board interactions in 3D using React Three Fiber, GSAP, and custom shaders.

3D CAPTURE ANIMATION SEQUENCE

1. ATTACK CHARGE —▶ Attacking piece lifts (Y-axis lift) and slides rapidly toward target tile. 2. IMPACT EVENT —▶ Collision generates particle explosion (stone debris/dust) & screen shake. 3. REMOVAL —▶ Captured piece model fractures/fades while physical piece is pulled to graveyard. 4. SETTLE —▶ Attacking piece lands on tile with heavy stone sound effect.

8. Implementation Roadmap

Phase	Deliverable	Key Milestones
Phase 1	Voice Engine & 3D Web App	Set up Web Speech parser, chess.js state engine, and Three.js 3D board rendering with move animations.
Phase 2	Gantry Mechanics & Firmware	Assemble CoreXY frame, write ESP32 G-code stepper pathfinder, and test electromagnet coupling force.
Phase 3	Sensory Matrix & Integration	Integrate 8x8 Hall-effect array, establish WebSocket bridge between web app and hardware, and tune audio/LED FX.
Phase 4	Enclosure & Marble Piece Fabrication	Finalize slate top surface, drill magnet cavities into marble piece bases, apply PTFE pads, and complete system testing.

Summary & Next Steps

This specification bridges software voice parsing, 3D browser graphics, and mechanical hardware engineering into a unified architecture. You can now build the software stack or order the motion control components.

